
$$\rightarrow \delta(q_0, 1) = (q_1, 0, D)$$

Devoir Maison

Labyrinthes, Machines de Turing et Complexité

Une Analyse Formelle : Modèles, Algorithmes et Complexité

Master 1 Informatique Avancée et Application (I2A)

Université Marie & Louis Pasteur, Franche-Comté

N'jie Zamon

Formation à distance en alternance
Depuis la Martinique

6 décembre 2025

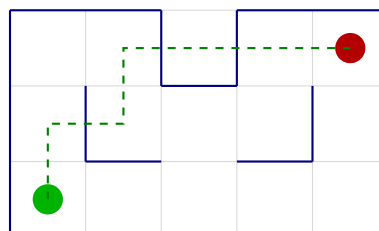


Table des matières

1	Formalisation d'un Labyrinthe	2
1.1	Illustration d'un labyrinthe simple	2
1.2	Modélisation par un graphe non orienté	2
2	Machines de Turing et Décidabilité	3
2.1	Stratégie d'encodage de la grille	3
2.2	Définition des états et des transitions	3
2.3	Analyse de la décidabilité	4
3	Algorithmes de Résolution de Labyrinthes	5
3.1	Étude comparative des algorithmes classiques	5
3.1.1	Algorithme de Pledge	5
3.1.2	Algorithme de Trémaux	5
3.1.3	Algorithme de Tarry	5
3.1.4	Algorithmes de parcours de graphe : DFS et BFS	5
3.2	Analyse comparative des complexités	6
3.3	Illustration de l'algorithme BFS	6
4	Complexité Algorithmique	6
4.1	Pseudo-code de l'algorithme BFS	6
4.2	Identification des opérations élémentaires	6
4.3	Détermination de la complexité asymptotique	7
5	Labyrinthes Enrichis de Contraintes	8
5.1	Formalisation du problème décisionnel	8
5.2	Appartenance à la classe NP	8
5.3	Démonstration de la NP-complétude et réduction	9
5.4	Implications pour la question P vs NP	9

1 Formalisation d'un Labyrinthe

1.1 Illustration d'un labyrinthe simple

Pour bien comprendre le concept, définissons un labyrinthe L basique, de taille 3×3 , caractérisé par :

- $C = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)\}$, qui représente l'ensemble des cases accessibles.
- $s = (1, 1)$, désignée comme la case initiale.
- $f = (3, 3)$, qui est la case finale à atteindre.

Dans ce scénario simplifié, nous considérons que toutes les cases sont libres, c'est-à-dire qu'il n'y a aucun mur. Un trajet possible pour relier s à f serait :

$$(1, 1) \rightarrow (1, 2) \rightarrow (1, 3) \rightarrow (2, 3) \rightarrow (3, 3).$$

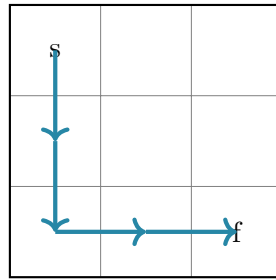


FIGURE 1 – Visualisation du labyrinthe 3x3 et d'un chemin possible.

1.2 Modélisation par un graphe non orienté

La question de la résolution d'un labyrinthe se transpose élégamment en un problème de parcours de graphe. Un labyrinthe $L = (C, s, f)$ peut être représenté par un graphe non orienté $G = (V, A)$ où :

- Les sommets V correspondent aux cases libres C .
- Les arêtes A connectent les sommets dont les cases sont adjacentes dans le labyrinthe.

La relation d'adjacence entre deux cases $u = (i, j)$ et $v = (i', j')$ est formellement définie par :

$$u \sim v \iff |i - i'| + |j - j'| = 1.$$

Ainsi, l'ensemble des arêtes est $A = \{\{u, v\} \in C \mid u \sim v\}$.

Par conséquent, trouver un chemin de s à f dans le labyrinthe est rigoureusement équivalent à identifier un chemin du sommet s au sommet f dans le graphe G . Il s'agit d'un problème fondamental de la théorie des graphes, pour lequel des algorithmes standards comme le parcours en largeur (BFS) ou en profondeur (DFS) sont couramment employés.

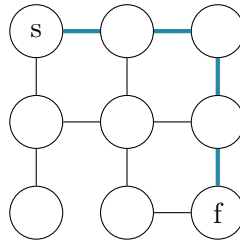


FIGURE 2 – Graphe non orienté équivalent au labyrinthe de la Figure 1.

2 Machines de Turing et Décidabilité

2.1 Stratégie d'encodage de la grille

Pour permettre à une machine de Turing (MT) d'interagir avec un labyrinthe, il est indispensable de définir un encodage de ce dernier sur son ruban. Voici une approche possible :

Alphabet d'entrée Σ : $\Sigma = \{0, 1, \#, s, f\}$

- 0 : représente une case murée (inaccessible).
- 1 : symbolise une case libre.
- # : agit comme un séparateur entre les lignes du labyrinthe.
- s : marque la position de départ.
- f : marque la position d'arrivée.

Alphabet du ruban Γ : $\Gamma = \Sigma \cup \{M, N, P, B\}$

- M, N, P : symboles de travail utilisés pour marquer les cases visitées durant l'exploration (par exemple, M pour les cases en cours d'analyse).
- B : le symbole blanc, représentant une case non utilisée du ruban.

Encodage du labyrinthe : Un labyrinthe de dimensions $m \times n$ est encodé comme une séquence de m lignes, chacune contenant n symboles, les lignes étant délimitées par #. Le labyrinthe de la Figure 1 serait donc représenté sur le ruban par la chaîne : **s11#111#11f#**

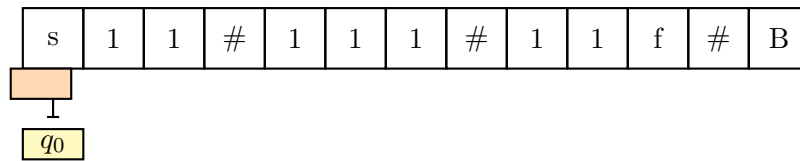


FIGURE 3 – Encodage du labyrinthe sur le ruban d'une machine de Turing.

2.2 Définition des états et des transitions

La machine de Turing $M = (Q, \Sigma, \Gamma, \delta, q_0, q_f)$ est conçue pour simuler un algorithme de recherche de chemin, s'apparentant à un DFS ou un BFS.

États principaux de la machine :

- q_0 : état initial, dont le rôle est de localiser le point de départ s .
- $q_{exploration}$: état central qui gère l'exploration des cases adjacentes.
- $q_{retour_arriere}$: état activé lorsqu'une impasse est détectée, pour revenir au point de décision précédent.
- q_{accept} : état final qui confirme la découverte d'un chemin vers f .
- q_{reject} : état final qui conclut à l'absence de chemin après exploration exhaustive.

Exemples de transitions de la fonction δ :

- Dans l'état q_0 , si le symbole lu est s , la MT le remplace par M (marquage) et transitionne vers $q_{exploration}$.
- Dans $q_{exploration}$, si une case libre 1 non marquée est rencontrée, elle est marquée M , sa position est mémorisée (potentiellement sur un ruban auxiliaire) et la tête se déplace vers elle.
- Si, depuis $q_{exploration}$, toutes les cases voisines sont des murs (0) ou déjà visitées (M), la machine passe à $q_{retour_arriere}$.

— Si f est lu depuis $q_{exploration}$, la machine transitionne à q_{accept} .

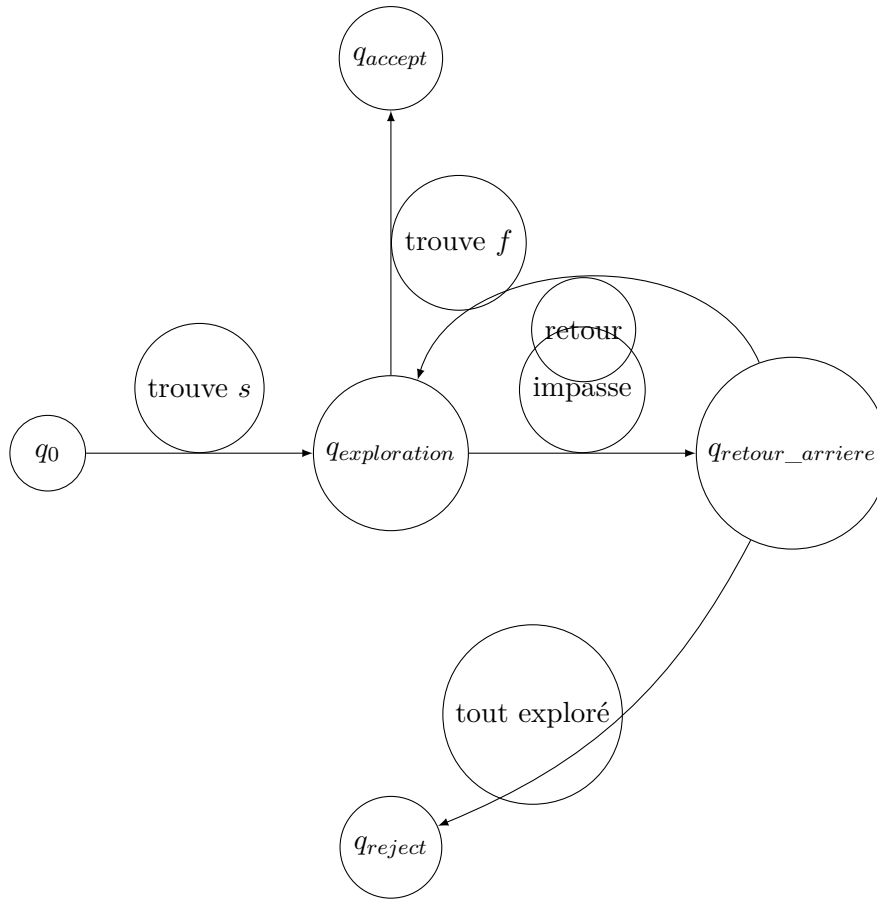


FIGURE 4 – Diagramme d'états simplifié pour la machine de Turing de résolution.

2.3 Analyse de la décidabilité

Cas d'un labyrinthe fini : Le problème décisionnel "Existe-t-il un chemin de s à f ?" est **décidable**. *Justification :* Un labyrinthe fini contient un nombre fini de cases et, par conséquent, un nombre fini de chemins potentiels. Une machine de Turing peut explorer systématiquement toutes les possibilités (par exemple, via une stratégie de type DFS). L'espace d'états étant fini, la machine est assurée de s'arrêter : soit elle trouvera un chemin et atteindra q_{accept} , soit elle épuisera toutes les options sans succès et aboutira à q_{reject} .

Cas d'un labyrinthe potentiellement infini : Le problème devient **indécidable** dans sa généralité. *Justification :* Si le labyrinthe est infini, l'espace des chemins à explorer l'est aussi. Une machine de Turing pourrait s'engager dans un chemin de longueur infinie sans jamais trouver f , et donc ne jamais s'arrêter. Il devient impossible de distinguer un calcul qui ne s'arrête pas car il n'y a pas de solution, d'un calcul qui ne s'arrête pas car le chemin est infiniment long. Cette problématique est intimement liée au problème de l'arrêt, connu pour être indécidable.

3 Algorithmes de Résolution de Labyrinthes

3.1 Étude comparative des algorithmes classiques

3.1.1 Algorithme de Pledge

- **Objectif** : Conçu spécifiquement pour les labyrinthes contenant des îlots (murs non connectés au périmètre extérieur). L'algorithme avance jusqu'à rencontrer un mur, puis le longe. Un compteur de rotations est maintenu : +1 pour un quart de tour à gauche, -1 pour un quart de tour à droite.
- **Méthodologie** : Quand le compteur de rotation retourne à zéro, l'algorithme "détache" du mur et reprend sa direction initiale. Cette stratégie garantit qu'il ne tournera pas indéfiniment autour du même îlot.
- **Inconvénients** : Il n'offre aucune garantie sur le caractère optimal du chemin trouvé. Son efficacité peut être fortement réduite si l'entrée et la sortie sont situées sur des îlots distincts.

3.1.2 Algorithme de Trémaux

- **Objectif** : Un algorithme de parcours systématique qui utilise un marquage pour éviter de revisiter les mêmes chemins et de tomber dans des cycles.
- **Méthodologie** : À chaque intersection, l'algorithme choisit une voie non marquée. Les passages empruntés sont marqués. En cas d'impasse, il fait demi-tour. Si une intersection déjà visitée est atteinte, il emprunte une autre voie si possible.
- **Inconvénients** : Il requiert un espace mémoire pour stocker les marques. Le chemin trouvé n'est pas nécessairement le plus court en termes de distance ou de nombre d'étapes.

3.1.3 Algorithme de Tarry

- **Objectif** : Similaire à Trémaux, mais avec une contrainte plus stricte : chaque arête (passage entre deux cases) sera traversée au maximum deux fois (une fois dans chaque direction).
- **Méthodologie** : L'algorithme explore le labyrinthe de façon exhaustive. Il garantit que chaque sommet (case) accessible sera visité.
- **Inconvénients** : L'exploration exhaustive peut s'avérer très lente avant de trouver la sortie, particulièrement dans des labyrinthes de grande taille.

3.1.4 Algorithmes de parcours de graphe : DFS et BFS

DFS (Depth-First Search) :

- **Objectif** : Explore le labyrinthe en profondeur, en suivant une branche aussi loin que possible avant de procéder à un retour en arrière (backtracking).
- **Méthodologie** : Typiquement implémenté à l'aide d'une pile (stack).
- **Inconvénients** : Ne garantit pas la découverte du chemin le plus court en nombre d'étapes.

BFS (Breadth-First Search) :

- **Objectif** : Explore le labyrinthe en largeur, couche par couche, à partir du point de départ.
- **Méthodologie** : Généralement implémenté avec une file (queue).
- **Inconvénients** : Consomme plus de mémoire que le DFS, car la file peut contenir de nombreux nœuds à la frontière de l'exploration. Cependant, il garantit de trouver le chemin le plus court (en nombre d'étapes).

3.2 Analyse comparative des complexités

TABLE 1 – Comparaison des complexités pour les algorithmes de résolution

Algorithme	Complexité temporelle	Complexité spatiale
Pledge	$O(C + A)$	$O(C)$
Trémaux	$O(C + A)$	$O(C + A)$
Tarry	$O(C + A)$	$O(C + A)$
DFS	$O(C + A)$	$O(C)$
BFS	$O(C + A)$	$O(C)$

Ici, $|C|$ représente le nombre de cases libres (sommets) et $|A|$ le nombre d'arêtes. La complexité temporelle est linéaire par rapport à la taille du graphe, car chaque sommet et chaque arête est visité un nombre constant de fois. La complexité spatiale dépend de la structure de données utilisée (pile ou file pour DFS/BFS, marques pour Trémaux/Tarry).

3.3 Illustration de l'algorithme BFS

Appliquons l'algorithme BFS au labyrinthe de la Figure 1 pour illustrer son fonctionnement.

4 Complexité Algorithmique

4.1 Pseudo-code de l'algorithme BFS

Algorithme 1 : Algorithme BFS pour la recherche de chemin dans un labyrinthe.

Input : Un labyrinthe $G = (V, A)$, un sommet de départ s , un sommet d'arrivée f
Output : VRAI si un chemin existe, FAUX sinon
 créer une file F ;
 créer un ensemble de sommets visités $Visites$;
 marquer s comme visité, $Visites \leftarrow Visites \cup \{s\}$;
 ajouter s à F ;
while F n'est pas vide **do**
 $u \leftarrow$ retirer le premier élément de F ;
 if $u = f$ **then**
 return VRAI ; // chemin trouvé
 end
 for chaque voisin v de u **do**
 si $v \notin Visites$ **alors**
 marquer v comme visité, $Visites \leftarrow Visites \cup \{v\}$;
 ajouter v à F ;
 fin
 end
end
return FAUX ; // aucun chemin trouvé

4.2 Identification des opérations élémentaires

Les opérations fondamentales dont le coût est constant, c'est-à-dire en $O(1)$, sont :

1. Accès et modification d'un élément dans la file F (enqueue, dequeue).
2. Test d'appartenance et ajout dans l'ensemble des visités $Visites$.

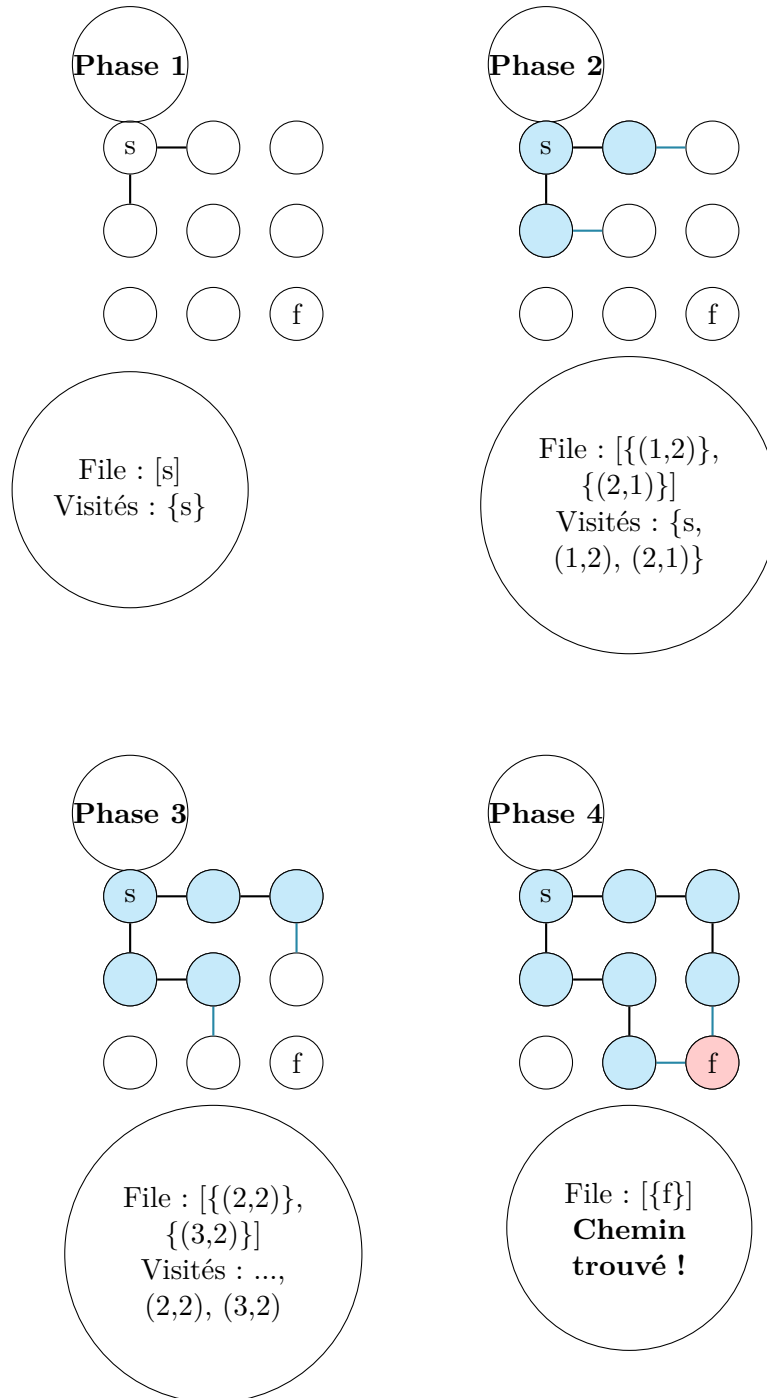


FIGURE 5 – Illustration pas à pas de l'algorithme BFS.

3. Accès à la liste des voisins d'un sommet.
4. Comparaison de sommets.

4.3 Détermination de la complexité asymptotique

Pour un labyrinthe modélisé par un graphe avec $|V|$ sommets (cases libres) et $|A|$ arêtes :

Complexité temporelle : $O(|V| + |A|)$ *Analyse détaillée :*

- Chaque sommet est retiré de la file au plus une seule fois. Le coût associé est donc de $O(|V|)$.

5 LABYRINTHES ENRICHIS DE CONTRAINTES

- Pour chaque sommet u retiré, la liste de ses voisins est parcourue. Sur l'ensemble de l'algorithme, chaque arête (u, v) sera examinée à deux reprises : une fois lors du traitement de u , et une fois lors de celui de v . Le coût total lié aux arêtes est donc de $O(|A|)$.
- Le coût global est la somme de ces deux contributions : $O(|V|) + O(|A|) = O(|V| + |A|)$.

Complexité spatiale : $O(|V|)$ *Analyse détaillée :*

- L'ensemble des sommets visités $Visites$ peut contenir, au pire, tous les sommets du graphe, soit $|V|$ éléments.
- La file F peut, dans le scénario le plus défavorable (pour un graphe très "large"), contenir un nombre de sommets proportionnel à $|V|$.
- La complexité spatiale est donc dominée par le stockage des sommets, ce qui nous donne $O(|V|)$.

5 Labyrinthes Enrichis de Contraintes

5.1 Formalisation du problème décisionnel

Problème : "Existe-t-il un chemin de s à f qui respecte l'ensemble des contraintes imposées?"

Définition formelle : Un labyrinthe à contraintes est défini par le sextuplé $L = (V, s, f, P, C, T)$ où :

- V est l'ensemble des cases (sommets) libres.
- $s \in V$ est le sommet de départ.
- $f \in V$ est le sommet d'arrivée.
- $P \subseteq V \times K$ est l'ensemble des *portes*, où chaque porte $p \in P$ est verrouillée par une clé $k \in K$.
- $K \subseteq V$ est l'ensemble des *clés* disséminées dans le labyrinthe.
- $T \subseteq V \times \mathbb{N}$ est l'ensemble des *contraintes temporelles*, où chaque case $(v, t) \in T$ doit être atteinte en moins de t étapes.

Question : Existe-t-il un chemin $p = (v_0, v_1, \dots, v_n)$ tel que :

- $v_0 = s$ et $v_n = f$.
- Pour tout i , $(v_i, v_{i+1}) \in A$ (arêtes du graphe).
- Pour toute porte $(p, k) \in P$ traversée à l'étape i , la clé k a été collectée dans une case v_j précédente ($j < i$).
- Pour toute contrainte temporelle $(v, t) \in T$, si $v = v_i$ alors $i \leq t$.

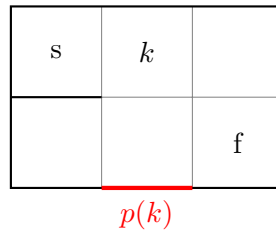


FIGURE 6 – Exemple de labyrinthe à contraintes : une porte p nécessitant la clé k .

5.2 Appartenance à la classe NP

Ce problème décisionnel appartient à la classe de complexité NP. Un problème est dans NP si une solution proposée (un certificat) peut être vérifiée en temps polynomial.

5 LABYRINTHES ENRICHIS DE CONTRAINTES

- **Certificat** : Le chemin $p = (v_0, \dots, v_n)$ lui-même. Sa taille, $n + 1$, est polynomiale par rapport à la taille de l'entrée (qui est $|V| + |P| + |K| + |T|$).
- **Vérification en temps polynomial** : Étant donné le chemin p , la vérification des contraintes peut être réalisée en un temps $O(n)$:
 1. Vérifier que $v_0 = s$ et $v_n = f$.
 2. S'assurer que chaque (v_i, v_{i+1}) est une arête valide du graphe.
 3. Contrôler les contraintes de clés : en parcourant le chemin, maintenir un ensemble des clés collectées. À chaque porte, vérifier que la clé requise est bien présente dans cet ensemble.
 4. Valider les contraintes temporelles : pour chaque sommet v_i , vérifier si $(v_i, t) \in T$ et si $i \leq t$.

5.3 Démonstration de la NP-complétude et réduction

Il est fort probable que ce problème soit NP-complet. Nous pouvons le démontrer par une réduction polynomiale depuis un problème connu comme étant NP-complet, tel que **HAMILTONIAN PATH**.

Réduction depuis HAMILTONIAN PATH :

- **Entrée** : Un graphe $G = (V, A)$ et deux sommets distincts $s, t \in V$. La question est : existe-t-il un chemin hamiltonien de s à t (un chemin qui visite chaque sommet exactement une fois) ?
- **Transformation** : Construisons une instance L de notre problème de labyrinthe à contraintes :
 - Les cases du labyrinthe sont les sommets V . Les arêtes sont A .
 - Le départ est s et l'arrivée est $f = t$.
 - Il n'y a ni portes ni clés ($P = K = \emptyset$).
 - On ajoute une contrainte temporelle sur chaque sommet : $T = \{(v, |V| - 1) \mid v \in V\}$.
- **Analyse de la transformation** : La construction de L à partir de G s'effectue en un temps polynomial en $|V|$.
- **Preuve de la réduction** :
 - $(\Rightarrow)$ Si G possède un chemin hamiltonien de s à t , ce chemin a une longueur de $|V| - 1$. Il visite chaque sommet exactement une fois. Par conséquent, pour chaque sommet v_i à l'étape i , on a $i \leq |V| - 1$. Le chemin respecte donc les contraintes temporelles de L .
 - $(\Leftarrow)$ Si L admet un chemin de s à f respectant les contraintes, alors ce chemin doit avoir une longueur $\leq |V| - 1$. Comme il doit relier s à t dans un graphe de $|V|$ sommets, et qu'il ne peut visiter deux fois le même sommet (sinon il dépasserait la longueur $|V| - 1$), il doit nécessairement visiter chaque sommet exactement une fois. C'est donc un chemin hamiltonien dans G .

Puisque **HAMILTONIAN PATH** est NP-complet, notre problème est au moins NP-difficile. Ayant prouvé qu'il est dans NP, nous concluons qu'il est **NP-complet**.

5.4 Implications pour la question P vs NP

- **Si $P = NP$** : Cela impliquerait l'existence d'un algorithme en temps polynomial pour résoudre ces labyrinthes à contraintes. Une telle découverte aurait des répercussions considérables dans de nombreux domaines (logistique, planification, etc.).
- **Si $P \neq NP$** : Cela confirmerait qu'il n'existe probablement pas d'algorithme en temps polynomial pour le cas général. Pour des labyrinthes de grande taille, il faudrait se tourner vers :

5 LABYRINTHES ENRICHIS DE CONTRAINTES

- Des algorithmes d'approximation ou des heuristiques (trouvant de "bons" chemins, sans garantie d'optimalité).
- Des algorithmes exponentiels (praticables pour des instances de petite taille).
- La résolution de sous-problèmes plus simples (par exemple, sans les contraintes temporelles).

La NP-complétude de ce problème le place au cœur des enjeux fondamentaux de l'informatique théorique.